Clasificación de Uso de Casco de Seguridad en Obras de Construcción mediante Transfer Learning con ResNet18

Manolo Sebastián Iñiguez Ramírez
Maestría en Inteligencia Artificial
Universidad San Francisco de Quito
msiniguezr@estud.usfq.edu.ec

Resumen

Se presenta un clasificador binario de imágenes capaz de distinguir si un trabajador de construcción usa casco de seguridad o no, utilizando Transfer Learning con ResNet18 pre-entrenada en ImageNet. El dataset proviene del *Construction Site Safety Image Dataset* de Kaggle, originalmente anotado en formato YOLO para detección de objetos, del cual se extrajeron 4 167 recortes de las clases *Hardhat* y *NO-Hardhat*. Se realizaron dos entrenamientos iterativos: el primero (V1) con parámetros iniciales alcanzó 94.03 % de accuracy en test, y tras un ajuste guiado por el análisis de las curvas de entrenamiento, el segundo (V2) mejoró a 96.18 % de accuracy, 96.03 % de F1-Score y solo 11 falsos negativos en 419 imágenes de test. Este clasificador funciona como baseline para una tesis de detección de Equipo de Protección Personal (EPP) con YOLO, validando que el enfoque de visión computacional es viable para este dominio.

1. Introducción y Trabajos Relacionados

La seguridad laboral en obras de construcción es un problema crítico. El casco de seguridad es el elemento de protección más importante para prevenir lesiones graves en la cabeza, y su uso correcto es obligatorio en la mayoría de normativas de seguridad laboral. Sin embargo, el monitoreo manual del cumplimiento es costoso, inconsistente y propenso a errores humanos.

En este contexto, los sistemas de visión computacional ofrecen una alternativa prometedora, cámaras conectadas a modelos de deep learning que detectan automáticamente si los trabajadores usan su equipo de protección personal (EPP). Este proyecto es parte de una tesis que propone desarrollar un sistema de este tipo, comenzando por la detección del casco de seguridad.

El objetivo del presente trabajo es construir un **baseline de clasificación binaria** (con casco / sin casco) que valide la viabilidad del enfoque antes de pasar a un sistema de detección completo con YOLO. La lógica es directa, si una CNN ni siquiera puede distinguir un recorte de casco versus no-casco, no tiene sentido construir un sistema más complejo.

1.1. Transfer Learning con CNNs

Transfer Learning es una técnica que consiste en tomar un modelo pre-entrenado en un dataset grande y reutilizar sus pesos como punto de partida para una tarea diferente. En lugar de aprender desde cero, el modelo ya tiene filtros útiles (bordes, texturas, formas) aprendidos del dataset original, lo cual acelera la convergencia y permite obtener buenos resultados incluso con datasets pequeños.

La biblioteca `torchvision` de PyTorch ofrece múltiples modelos pre-entrenados en ImageNet [3], entre ellos ResNet18. Esta arquitectura, propuesta por He et al. [1], introdujo las conexiones residuales (*skip connections*) que permiten entrenar redes más profundas sin el problema de degradación del gradiente. ResNet18 es la variante más ligera de la familia, con aproximadamente 11.2 millones de parámetros, lo que la hace ideal para fine-tuning en tareas de clasificación con datasets moderados.

Este mismo enfoque, fine-tuning de ResNet18 pre-entrenada en ImageNet, se aplicó en el Taller 01 del curso de Aprendizaje Profundo sobre el dataset SVHN [4], obteniendo 96.14 % de accuracy. El presente proyecto replica y adapta ese pipeline a un dominio real de seguridad laboral, con un dataset y

preprocesamiento completamente diferentes.

ImageNet [2] es un dataset de más de 14 millones de imágenes organizadas en 1 000 categorías, utilizado como estándar para pre-entrenar modelos de visión computacional. Los pesos aprendidos en ImageNet capturan representaciones visuales genéricas que se transfieren efectivamente a dominios específicos, como el de seguridad en construcción.

2. Metodología

2.1. Dataset y Preprocesamiento

Se utilizó el *Construction Site Safety Image Dataset* [5], publicado en Kaggle, que contiene 2 801 imágenes de obras de construcción anotadas en formato YOLO con 10 clases de objetos (cascos, chalecos, mascarillas, maquinaria, etc.).

Como el objetivo es clasificación binaria y no detección de objetos, se realizó un preprocesamiento para convertir las anotaciones YOLO a un dataset de clasificación, se leyeron las coordenadas de los bounding boxes de las clases 0 (*Hardhat*) y 2 (*NO-Hardhat*), se desnormalizaron las coordenadas, y se recortó cada región de la imagen original con un padding del 15 % para incluir contexto visual. Esto generó 4 167 recortes: 2 398 con casco (57.5 %) y 1 769 sin casco (42.5 %).

Los recortes se dividieron en tres conjuntos con un split estratificado por imagen original (para evitar fuga de datos entre splits): train (3 328 imágenes, 80 %), validation (420, 10 %) y test (419, 10 %).

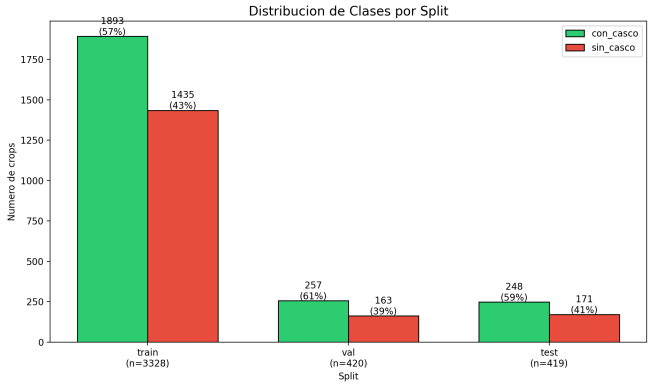

Figura 1: Distribución de clases por split. El split estratificado mantiene proporciones similares en train, validación y test.

2.2. Modelo

Se utilizó ResNet18 con pesos pre-entrenados de ImageNet, cargados desde `torchvision.models` [3]. La modificación clave fue reemplazar la capa fully connected final: la original (`Linear(512, 1000)` para las 1 000 clases de ImageNet) se sustituyó por una secuencia de `Dropout` + `Linear(512, 2)` para las dos clases del problema. Todos los pesos del backbone se entrenaron (fine-tuning completo).

El modelo se implementó como un `LightningModule` de PyTorch Lightning [6], con `CrossEntropyLoss` como función de pérdida, optimizador Adam, y métricas calculadas con `torchmetrics`: Accuracy, F1-Score, Precision y Recall.

Se aplicó data augmentation solo en el conjunto de entrenamiento: `RandomHorizontalFlip`, `RandomRotation(15)` y `ColorJitter` (brillo, contraste, saturación). Validación y test usaron únicamente `Resize(224)` y normalización con estadísticas de ImageNet.

2.3. Entrenamiento Iterativo

Se realizaron dos entrenamientos siguiendo un proceso de ajuste guiado por el análisis de las curvas:

V1 (entrenamiento base): LR=3e-4, Dropout=0.2, Patience=5, máximo 30 epochs. El modelo convergió rápido (mejor época: 4, val_loss=0.092) pero mostró inestabilidad en épocas tardías, en la época 7, el val_loss saltó a 0.159, casi el doble de su mejor valor.

V2 (ajuste informado): Tras analizar las curvas de V1, se redujo el learning rate a 1e-4 (para eliminar la inestabilidad), se subió el dropout a 0.4 (más regularización) y se incrementó el patience a 7 (porque con LR más bajo el modelo tarda más en converger). El modelo entrenó 22 épocas y alcanzó su mejor resultado en la época 15 (val_loss=0.050).

En ambos casos se utilizaron los callbacks `ModelCheckpoint` (guarda automáticamente el mejor modelo según val_loss) y `EarlyStopping`, con `CSVLogger` para registrar métricas por época.

3. Resultados y Discusión

3.1. Curvas de Entrenamiento

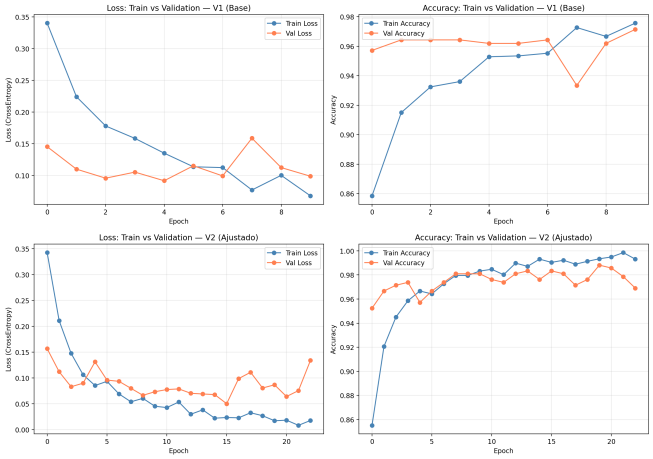

Figura 2: Curvas de entrenamiento. Arriba: V1 muestra convergencia rápida pero inestabilidad en época 7. Abajo: V2 converge de forma más gradual y estable.

Las curvas de V1 muestran una convergencia rápida (mejor época en la 4) pero con un salto abrupto de val_loss en la época 7 (de 0.092 a 0.159), indicando que el learning rate de 3e-4 era demasiado alto para el ajuste fino. V2, con LR reducido a 1e-4, mostró una convergencia más gradual y estable, alcanzando un val_loss de 0.050 en la época 15, prácticamente la mitad del mejor valor de V1.

3.2. Evaluación en Test

Se seleccionó automáticamente V2 como mejor modelo (menor val_loss) y se evaluaron ambos modelos en el conjunto de test (419 imágenes nunca vistas durante el entrenamiento). La Tabla 1 muestra la comparación:

Cuadro 1: Comparación de resultados V1 vs V2 en test.

Métrica	**V1**	**V2**
Mejor época	4	15
val_loss (mejor)	0.0918	0.0501
Test Accuracy	94.03 %	96.18 %
Test F1-Score	93.85 %	96.03 %
Test Precision	93.68 %	96.32 %
Test Recall	94.05 %	95.78 %

El ajuste de hiperparámetros mejoró todas las métricas. La mejora más notable fue en val_loss (de 0.092 a 0.050, reducción del 45 %), lo que se tradujo en un incremento de 2.15 puntos porcentuales en test accuracy. Esto confirma que el diagnóstico basado en las curvas fue correcto: la inestabilidad de V1 se debía al learning rate alto, y la solución fue reducirlo.

Cabe comparar estos resultados con el Taller 01 del curso, donde se aplicó el mismo enfoque (fine-tuning de ResNet18) sobre SVHN (10 clases de dígitos) y se obtuvo 96.14 % de accuracy. Nuestro clasificador binario alcanza un resultado comparable (96.18 %), lo cual es razonable dado que el problema es más simple (2 clases vs 10) pero el dataset es más pequeño y de un dominio muy diferente.

3.3. Análisis de la Matriz de Confusión

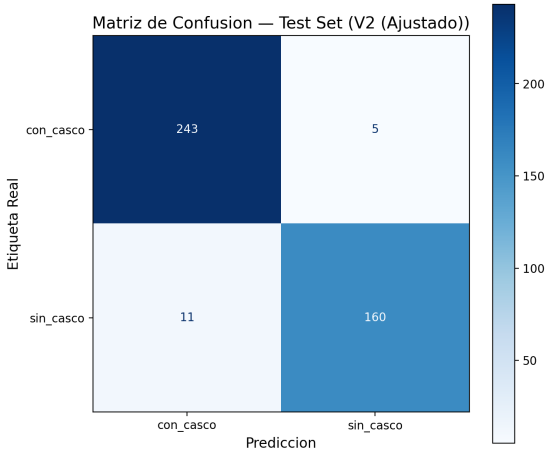

Figura 3: Matriz de confusión del modelo V2 en el conjunto de test. Los 11 falsos negativos representan el error crítico desde la perspectiva de seguridad laboral.

La matriz de confusión del modelo V2 (Figura 3) muestra 243 verdaderos negativos, 160 verdaderos positivos, 5 falsos positivos y 11 falsos negativos. El modelo comete solo 16 errores en 419 imágenes.

Desde la perspectiva de seguridad laboral, los **11 falsos negativos** son el error crítico, representan trabajadores sin casco que el modelo clasificó como si lo tuvieran. Esto equivale a un 6.4 % de error en la clase *sin_casco* (11 de 171). En un sistema real de monitoreo, estos casos pasarían desapercibidos, lo cual representa un riesgo.

Los 5 falsos positivos son menos preocupantes, solo generarían alertas innecesarias sin poner a nadie en peligro. Esta asimetría en la gravedad de los errores es un aspecto importante que se deberá considerar en la tesis al definir los umbrales de decisión del sistema de detección.

4. Conclusiones

Se entrenó un clasificador binario de imágenes que distingue la presencia o ausencia de casco de seguridad con 96.18 % de accuracy, 96.03 % de F1-Score y solo 16 errores en 419 imágenes de test, utilizando Transfer Learning con ResNet18 pre-entrenada en ImageNet.

El proceso de ajuste iterativo de hiperparámetros, guiado por el análisis de las curvas de entrenamiento, no por búsqueda a ciegas, demostró ser efectivo, reducir el learning rate de 3e-4 a 1e-4 y aumentar el dropout de 0.2 a 0.4 mejoró el val_loss en un 45 % y el test accuracy en 2.15 puntos porcentuales.

Limitaciones: El modelo clasifica recortes individuales, no imágenes completas de obra. Solo detecta un tipo de EPP (cascos) y el dataset puede no representar todas las condiciones reales de obras en Ecuador. El desbalance de clases (57 % con_casco vs 43 % sin_casco) podría estar contribuyendo a los 11 falsos negativos.

Trabajo futuro: Este clasificador valida la idea central de la tesis: si una CNN puede distinguir con más del 96 % si un recorte tiene casco o no, entonces tiene sentido construir un sistema completo de detección de EPP con YOLO. Los siguientes pasos incluyen pasar de clasificación a detección de objetos, incorporar más tipos de EPP (chalecos, guantes, gafas), y entrenar con datos reales de obras en Ecuador.

Técnicas del curso aplicadas: CNNs con Transfer Learning (Taller 01), data augmentation, PyTorch Lightning, early stopping, model checkpointing, métricas con `torchmetrics`, análisis de curvas de entrenamiento, y ajuste iterativo de hiperparámetros.

Referencias

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770–778, 2016.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 248–255, 2009.

[3] PyTorch, "Models and pre-trained weights — torchvision," [Online]. Available: `https://pytorch.org/vision/stable/models.html`. Accessed: Mar. 2026.

[4] PyTorch, "Datasets — torchvision," [Online]. Available: `https://docs.pytorch.org/vision/main/datasets.html`. Accessed: Mar. 2026.

[5] S. Sanyal, "Construction Site Safety Image Dataset," Kaggle, [Online]. Available: `https://www.kaggle.com/datasets/snehilsanyal/construction-site-safety-image-dataset-roboflow`. Accessed: Mar. 2026.

[6] Lightning AI, "PyTorch Lightning Documentation," [Online]. Available: `https://lightning.ai/docs/pytorch/stable/`. Accessed: Mar. 2026.